TP — Ingeniería de Software

Módulo de Seguridad

SistemaTicket-BE · Implementación paso a paso

Usuario, autenticación con hash BCrypt, gestión de sesiones (Singleton)
y permisos jerárquicos (Composite), persistidos con Spring Data JPA sobre H2.

Índice de pasos

Paso	Entregable	Capa
1	Entidad `User` (username/passwordHash) + repositorio	entities / data
2	`CryptoManager` (hash y comparación BCrypt manual)	business
3	`UserService.login()`	business
4	`SessionManager` — Singleton thread-safe	business
5	`AuthController` — POST /login, POST /logout	presentation
6	Composite de permisos: `PermissionComponent` / `Permission` / `PermissionGroup`	entities / data
7	Relación User↔Permisos + `PermissionService.hasPermission()`	entities / business

Paso 1 — Entidad User y repositorio

entities/User.java data/UserRepository.java

> Se extendió la entidad `User` ya existente (en vez de crear una entidad separada de credenciales) agregando `username` (único) y `passwordHash`. El campo se llama explícitamente **passwordHash** — nunca se guarda el password en texto plano; el hasheo lo hace `CryptoManager` en el paso 2.

entities/User.java

```
@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String firstName;
    private String lastName;

    @Column(unique = true, nullable = false)
    private String username;

    @Column(nullable = false)
    private String passwordHash;

    // getters / setters ...
}
```

data/UserRepository.java

```
public interface UserRepository extends JpaRepository<User, Long> {
    Optional<User> findByUsername(String username);
}
```

> **Por qué así:** `findByUsername` es un método derivado de Spring Data — genera una query parametrizada internamente, evitando concatenación de strings y por lo tanto SQL injection. H2 no requirió configuración extra: con `ddl-auto=update`, Hibernate agrega las columnas nuevas a la tabla `users` automáticamente al levantar la app.

Paso 2 — CryptoManager (hash BCrypt manual)

business/CryptoManager.java | pom.xml (+jbcrypt) | test/CryptoManagerTest.java

> Se agregó la dependencia `org.mindrot:jbcrypt` (librería standalone de BCrypt, **sin relación con Spring Security**) y se implementó una clase propia que la envuelve. BCrypt genera y embebe el salt dentro del propio hash, por eso no hace falta guardarlo aparte.

business/CryptoManager.java

```
@Component
public class CryptoManager {

    public String hash(String password) {
        return BCrypt.hashpw(password, BCrypt.gensalt());
    }

    public boolean compare(String plainPassword, String hash) {
        return BCrypt.checkpw(plainPassword, hash);
    }
}
```

terminal — mvn test (CryptoManagerTest)

```
-------------------------------------------------------------------------------
Test set: org.pablo.business.CryptoManagerTest
-------------------------------------------------------------------------------
Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.269 s
```

> **Por qué así:** se pidió explícitamente evitar `PasswordEncoder` de Spring Security. jBCrypt da el algoritmo probado (cost factor + salt) sin traer el resto del framework de seguridad. Los 3 tests cubren: el hash difiere del password plano, `compare()` valida el password correcto y rechaza uno incorrecto.

Paso 3 — UserService.login()

business/UserService.java test/UserServiceTest.java

business/UserService.java

```
public Optional<User> login(String username, String password) {
    return userRepository.findByUsername(username)
            .filter(user -> cryptoManager.compare(password, user.getPasswordHash()));
}
```

terminal — mvn test (UserServiceTest)

```
-------------------------------------------------------------------------------
Test set: org.pablo.business.UserServiceTest
-------------------------------------------------------------------------------
Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.600 s
```

Decisión de diseño — tipo de retorno: se eligió `Optional<User>` en vez de lanzar una excepción o crear un Result object a medida. Login inválido es un resultado normal de negocio, no un caso excepcional; además `Optional` ya era el patrón usado en `findById`, y el controller lo mapea directo a un HTTP status sin try/catch.

Decisión de seguridad: tanto "usuario no existe" como "password incorrecto" devuelven `Optional.empty()` — no se distinguen, para no permitir enumeración de usernames válidos por fuerza bruta.

Paso 4 — SessionManager (Singleton thread-safe)

business/SessionManager.java test/SessionManagerTest.java

> Antes de codear se analizó el trade-off entre **(a)** una sesión global única (apto para desktop mono-usuario) y **(b)** un mapa de sesiones por token (apto para Web API multiusuario concurrente). Se eligió **(b)**: el proyecto es explícitamente una Web API que atiende múltiples usuarios al mismo tiempo, y una sesión global haría que el login del Usuario B pise la sesión del Usuario A vista por otro request concurrente.

business/SessionManager.java

```
public class SessionManager {

    private static volatile SessionManager instance;
    private final Map<String, User> activeSessions = new ConcurrentHashMap<>();

    private SessionManager() {}

    public static SessionManager getInstance() {
        if (instance == null) {
            synchronized (SessionManager.class) {
                if (instance == null) {
                    instance = new SessionManager();
                }
            }
        }
        return instance;
    }

    public String createSession(User user) {
        String token = UUID.randomUUID().toString();
        activeSessions.put(token, user);
        return token;
    }

    public Optional<User> getSession(String token) {
        return Optional.ofNullable(activeSessions.get(token));
    }

    public void invalidateSession(String token) {
        activeSessions.remove(token);
    }
}
```

terminal — mvn test (SessionManagerTest, 50 threads concurrentes)

```
-------------------------------------------------------------------------------
Test set: org.pablo.business.SessionManagerTest
-------------------------------------------------------------------------------
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.005 s
> getInstanceReturnsTheSameInstanceUnderConcurrentAccess: 50 threads,
  1 sola instancia observada en el Set resultante.
```

> **Por qué double-checked locking y no `synchronized` en todo el método:** sincronizar todo `getInstance()` serializaría cada llamada aunque la instancia ya exista. El patrón paga el costo de sincronización una sola vez (la creación); después la comprobación externa es una simple lectura de un campo `volatile`. `volatile` es imprescindible para evitar que, por reordenamiento de instrucciones de la JVM, un thread vea una referencia no-null apuntando a un objeto a medio construir.

Paso 5 — Endpoints REST de login/logout

presentation/AuthController.java | presentation/LoginRequest.java | presentation/LoginResponse.java

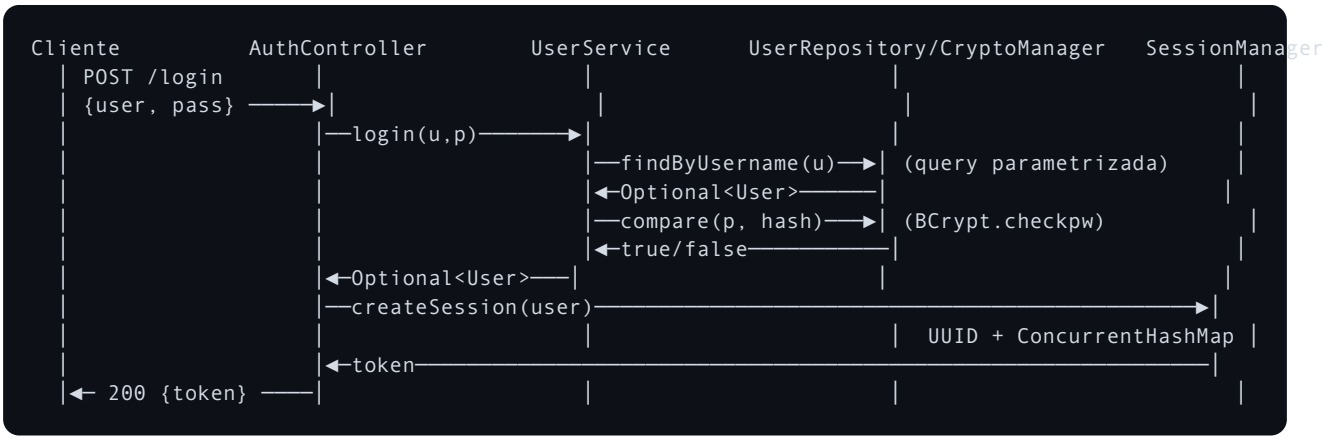
> **Qué devuelve el login:** el token de sesión generado por `SessionManager.createSession()`, en el body JSON — nunca el objeto `User` completo (evita filtrar el `passwordHash`). El cliente reenvía ese token en `Authorization: Bearer <token>` en requests posteriores, incluido `/logout`.

presentation/AuthController.java

```
@PostMapping("/login")
public ResponseEntity<?> login(@RequestBody LoginRequest request) {
    return userService.login(request.getUsername(), request.getPassword())
            .map(this::buildLoginResponse)
            .orElseGet(() -> ResponseEntity.status(HttpStatus.UNAUTHORIZED)
                    .body(new ErrorResponse("Invalid credentials")));
}

@PostMapping("/logout")
public ResponseEntity<?> logout(
        @RequestHeader(value = "Authorization", required = false) String
authorizationHeader) {
    String token = extractToken(authorizationHeader);
    if (token == null) {
        return ResponseEntity.status(HttpStatus.BAD_REQUEST)
                .body(new ErrorResponse("Missing session token"));
    }
    SessionManager.getInstance().invalidateSession(token);
    return ResponseEntity.noContent().build();
}
```

Flujo completo de punta a punta



Paso 6 — Patrón Composite para permisos

entities/PermissionComponent.java entities/Permission.java entities/PermissionGroup.java data/PermissionComponentRepository.java

> **Componente:** JPA solo mapea clases, no interfaces — por eso `PermissionComponent` es una clase *abstracta* que declara `add()`, `remove()`, `getChildren()`, `operation()`. `Permission` (hoja) y `PermissionGroup` (compuesto) la extienden.
>
> **Persistencia — estrategia elegida:** `SINGLE_TABLE` (vs. `JOINED`): una sola tabla `permission_components` con columna discriminadora `component_type`. Con solo 2 subtipos sin campos exclusivos, `JOINED` agregaría joins innecesarios; `SINGLE_TABLE` además simplifica la relación autoreferenciada padre-hijo, que puede apuntar a "cualquier componente" sin importar el subtipo.
>
> **Relación hijos:** `@ManyToMany` (no `@OneToMany`) vía tabla `permission_group_children` — permite que una misma Familia/Patente se reutilice dentro de más de una Familia padre sin duplicarse (ej. "ABM_BASE" dentro de "ADMIN" y de "SOPORTE").

entities/PermissionComponent.java (abstracta)

```
@Entity
@Table(name = "permission_components")
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name = "component_type", discriminatorType = DiscriminatorType.STRING)
public abstract class PermissionComponent {

    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String code;

    public abstract void add(PermissionComponent component);
    public abstract void remove(PermissionComponent component);
    public abstract List<PermissionComponent> getChildren();
    public abstract List<String> operation();
}
```

entities/Permission.java (hoja) / PermissionGroup.java (compuesto)

```
@Entity @DiscriminatorValue("PERMISSION")
public class Permission extends PermissionComponent {
    public void add(PermissionComponent c) {
        throw new UnsupportedOperationException("A Permission cannot have children");
    }
    public List<String> operation() { return List.of(getCode()); }
    // remove/getChildren análogos ...
}

@Entity @DiscriminatorValue("GROUP")
public class PermissionGroup extends PermissionComponent {

    @ManyToMany(cascade = {CascadeType.PERSIST, CascadeType.MERGE})
```

```
    @JoinTable(name = "permission_group_children",
        joinColumns = @JoinColumn(name = "parent_id"),
        inverseJoinColumns = @JoinColumn(name = "child_id"))
    private List<PermissionComponent> children = new ArrayList<>();

    public void add(PermissionComponent c) { children.add(c); }

    public List<String> operation() {
        List<String> codes = new ArrayList<>();
        for (PermissionComponent child : children) {
            codes.addAll(child.operation());   // recursión
        }
        return codes;
    }
}
```

terminal — mvn test (schema H2 real generado por Hibernate)

```
Hibernate: create table permission_components (component_type varchar(31) not null,
    id bigint generated by default as identity, code varchar(255) not null, primary key
(id))
Hibernate: create table permission_group_children (parent_id bigint not null, child_id
bigint not null)
Hibernate: alter table if exists permission_components add constraint UK... unique (code)
Hibernate: alter table if exists permission_group_children add constraint FK...
    foreign key (child_id) references permission_components
Hibernate: alter table if exists permission_group_children add constraint FK...
    foreign key (parent_id) references permission_components

-------------------------------------------------------------------------------
Test set: org.pablo.data.PermissionComponentRepositoryTest
-------------------------------------------------------------------------------
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.938 s

Test set: org.pablo.entities.PermissionCompositeTest
-------------------------------------------------------------------------------
Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.002 s
```

Paso 7 — Relación User↔Permisos y verificación recursiva

entities/User.java (relación) · business/PermissionService.java · test/PermissionServiceTest.java

> **Relación elegida:** `@ManyToMany` (no `@OneToMany`): un usuario puede tener varios componentes asignados directamente, y un mismo componente (típicamente una Familia como "ADMIN") se comparte entre múltiples usuarios — modelo tipo RBAC.

entities/User.java (fragmento)

```
@ManyToMany
@JoinTable(
    name = "user_permission_components",
    joinColumns = @JoinColumn(name = "user_id"),
    inverseJoinColumns = @JoinColumn(name = "component_id")
)
private List<PermissionComponent> permissionComponents = new ArrayList<>();
```

business/PermissionService.java

```
@Service
public class PermissionService {

    public boolean hasPermission(User user, String permissionCode) {
        return user.getPermissionComponents().stream()
                .flatMap(component -> component.operation().stream())
                .anyMatch(permissionCode::equals);
    }
}
```

> **Por qué no se reescribió la recursión acá:** `operation()` ya recorre recursivamente las Familias anidadas (paso 6). Reutilizarlo evita duplicar la lógica de recorrido del árbol en dos lugares distintos.

terminal — mvn test (PermissionServiceTest — búsqueda recursiva a 2 niveles)

```
-------------------------------------------------------------------------------
Test set: org.pablo.business.PermissionServiceTest
-------------------------------------------------------------------------------
Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.002 s
> userWithANestedFamilyContainingThePermissionHasItRecursively: OK
   (ADMIN → ABM_BASE → Permission "ABM_USUARIOS", encontrado en 2do nivel)
> userWithoutTheRequestedPermissionDoesNotHaveIt: OK
```

Build final — suite completa

```
terminal — mvn test (13 tests, módulo completo)

[INFO] Running org.pablo.business.UserServiceTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.600 s
[INFO] Running org.pablo.business.PermissionServiceTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.002 s
[INFO] Running org.pablo.business.SessionManagerTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.005 s
[INFO] Running org.pablo.business.CryptoManagerTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.269 s
[INFO] Running org.pablo.data.PermissionComponentRepositoryTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.938 s
[INFO] Running org.pablo.entities.PermissionCompositeTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.002 s

[INFO] Tests run: 13, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

Qué quedó pendiente / fuera de alcance

- No hay filtro/interceptor que exija el token de sesión en endpoints protegidos (login/logout gestionan el ciclo de vida, pero nada lo está exigiendo aún).
- `hasPermission()` todavía no está conectado a ningún endpoint concreto.
- `/logout` no valida que el token exista antes de invalidar (idempotente, sin error).
- No hay expiración/TTL de sesiones en `SessionManager`.
- Sin protección contra ciclos en el árbol de `PermissionGroup`.
- `ddl-auto=update` sin migraciones versionadas (Flyway/Liquibase).

Documento generado a partir de la implementación real del módulo — código y salidas de test corresponden al estado actual del repositorio SistemaTicket-BE.